



Data Management with SQL Server: Sessions 7–9 (Microsoft Azure SQL, Accessing Data, Advanced Queries, and Joins)

This document provides a comprehensive class note for Sessions 7, 8, and 9 of the "Data Management with SQL Server" course (OV-1507). It covers Microsoft Azure SQL, querying data with SELECT statements, practical exercises, advanced queries, joins, views, stored procedures, and metadata querying. The note includes step-by-step explanations, examples, classwork, and a test to reinforce understanding. Additionally, a section on creating an Azure account is included to support hands-on learning.

Session 7: Microsoft Azure SQL and Accessing Data

Overview

This session introduces Microsoft Azure SQL, a cloud-based relational database service, and covers querying data using the SELECT statement, including clauses and XML data handling. It also includes steps to create an Azure account for practical application.

Creating an Azure Account

Theory: Microsoft Azure is a cloud platform offering services like Azure SQL Database. To use Azure SQL, you need an Azure account.

Step-by-Step Guide:

1. **Visit Azure Portal:** Go to portal.azure.com.
2. **Sign Up:**
 - Click "Start free" or "Create account."
 - Sign in with a Microsoft account (or create one).
3. **Provide Details:**

- Enter personal information (name, email, phone).
 - Provide a valid credit card for verification (free tier available with no initial charge).
4. **Agree to Terms:** Accept the Azure subscription agreement.
 5. **Access Free Tier:**
 - Select the free tier (e.g., \$200 credit for 30 days, free services like Azure SQL Database for limited usage).
 6. **Verify Account:** Complete phone/email verification.
 7. **Access Portal:** Once verified, log in to the Azure Portal to create and manage resources.

Note: Use the free tier for educational purposes to avoid costs. Always monitor usage in the Azure Portal.

Classwork 7.1: Create an Azure account using the free tier. Take a screenshot of the Azure Portal dashboard after login and share it with the instructor.

Introduction to Azure SQL

Theory:

- **Azure SQL:** A fully managed relational database service based on SQL Server, offering scalability, high availability, and automated maintenance.
- **Key Differences from On-Premises SQL Server:**
 - **Deployment:** Cloud-hosted (Azure SQL) vs. local infrastructure (on-premises).
 - **Management:** Automated backups, patching (Azure SQL) vs. manual management (on-premises).
 - **Cost:** Pay-as-you-go (Azure SQL) vs. upfront hardware costs (on-premises).
- **Benefits:**
 - **Scalability:** Adjust compute/storage on-demand.
 - **Availability:** 99.99% uptime with geo-replication.
 - **Security:** Built-in encryption, compliance (e.g., GDPR, HIPAA).

Example: Deploy an Azure SQL Database.

1. In Azure Portal, click "Create a resource" > "SQL Database."
2. Configure:
 - Database Name: SalesDB
 - Server: Create a new server (e.g., salesdb-server).

- Authentication: SQL Authentication (set admin username/password).
 - Pricing Tier: Select "Basic" for free tier or educational use.
3. Deploy and note the server name (e.g., `salesdb-server.database.windows.net`).

Connecting to Azure SQL with SSMS

Theory: SQL Server Management Studio (SSMS) is used to manage and query Azure SQL databases.

Step-by-Step Guide:

1. **Download SSMS:** Install SSMS from [Microsoft's website](#) if not already installed.
2. **Open SSMS:** Launch SSMS.
3. **Enter Server Details:**
 - Server Name: `<your-server-name>.database.windows.net` (e.g., `salesdb-server.database.windows.net`).
 - Authentication: SQL Server Authentication.
 - Username/Password: Use the admin credentials set during server creation.
4. **Configure Firewall:**
 - In Azure Portal, navigate to the SQL server > "Firewalls and virtual networks."
 - Add your client IP address to allow access.
5. **Connect:** Click "Connect" in SSMS.
6. **Verify:** Use Object Explorer to view databases (e.g., `SalesDB`).

Example:

```
-- Connect to SalesDB on Azure
-- Server: salesdb-server.database.windows.net
-- Authentication: SQL Server Authentication
-- Username: admin_user
-- Password: *****
```

SELECT Statement and Clauses

Theory:

- The SELECT statement retrieves data from tables.

- **Syntax:**

```
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column [ASC|DESC];
```

- **Clauses:**

- **WHERE:** Filters rows (e.g., `WHERE Age > 30`).
- **GROUP BY:** Groups rows for aggregation (e.g., `GROUP BY Department`).
- **HAVING:** Filters grouped data (e.g., `HAVING COUNT(*) > 5`).
- **ORDER BY:** Sorts results (e.g., `ORDER BY Salary DESC`).

Example: Retrieve employees with salaries above 50,000, sorted by name.

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary > 50000
ORDER BY FirstName ASC;
```

Working with XML

Theory:

- **Typed XML:** Uses an XML schema for validation.
- **Untyped XML:** No schema, more flexible.
- **Use Case:** Store and query hierarchical or semi-structured data.

Example: Generate XML from Employees table.

```
SELECT FirstName, LastName
FROM Employees
FOR XML PATH('Employee');
```

Output:

```
<Employee>
  <FirstName>John</FirstName>
  <LastName>Doe</LastName>
</Employee>
```

Creating XML Schema:

```
CREATE XML SCHEMA COLLECTION EmployeeSchema AS
N'<?xml version="1.0" encoding="UTF-16"?>
<schema xmlns="http://www.w3.org/2001/XMLSchema">
  <element name="Employee">
    <complexType>
      <sequence>
        <element name="FirstName" type="string"/>
        <element name="LastName" type="string"/>
      </sequence>
    </complexType>
  </element>
</schema>';
```

Classwork 7.2: Using the AdventureWorks2022 database on Azure SQL:

1. Connect to the database using SSMS.
2. Write a SELECT query to retrieve products where `UnitPrice > 100`, sorted by `ProductName`.
3. Modify the query to output results as XML using `FOR XML PATH`.

Session 8: Try It Yourself (Sessions 5–8)

Theory: This session reinforces concepts from database creation, table management, and querying through hands-on exercises.

Practical Exercises

1. Database Creation:

- Create a database named `TestDB`.

```
CREATE DATABASE TestDB;
```

2. Table Management:

- Create a `Customers` table with columns: `CustomerID` (INT, primary key), `FirstName` (NVARCHAR(50)), `LastSecond` (NVARCHAR(50)).

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    FirstName NVARCHAR(50),  
    LastSecond NVARCHAR(50)  
);
```

3. SELECT Query:

- Retrieve all customers, ordered by `LastSecond`.

```
SELECT FirstName, LastSecond  
FROM Customers  
ORDER BY LastSecond ASC;
```

Classwork 8.1:

1. In `TestDB`, create an `Orders` table with columns: `OrderID` (INT, primary key), `CustomerID` (INT), `OrderDate` (DATE).
2. Insert 3 sample rows:

```
INSERT INTO Orders (OrderID, CustomerID, OrderDate)  
VALUES  
    (1, 101, '2025-01-01'),  
    (2, 102, '2025-02-01'),  
    (3, 101, '2025-03-01');
```

3. Query the table, sorting by `OrderDate`.

```
SELECT OrderID, CustomerID, OrderDate  
FROM Orders  
ORDER BY OrderDate ASC;
```

Session 9: Advanced Queries and Joins

Theory: This session covers advanced querying techniques, including grouping, subqueries, joins, table expressions, set operators, views, stored procedures, and metadata querying.

Grouping and Aggregating Data

Theory:

- **GROUP BY:** Groups rows for aggregate functions (e.g., COUNT, SUM, AVG).
- **Aggregate Functions:**
 - **COUNT** : Counts rows.
 - **SUM** : Sums values.
 - **AVG** : Averages values.
- **HAVING:** Filters grouped results.

Example:

```
SELECT Department, COUNT(*) AS EmployeeCount, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;
```

Subqueries

Theory:

- **Non-Correlated Subquery:** Independent of the outer query.
- **Correlated Subquery:** References the outer query.

Examples:

- **Non-Correlated:**

```
SELECT ProductName
FROM Products
WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM Products);
```

- **Correlated:**

```
SELECT FirstName, LastName
FROM Employees e
WHERE EXISTS (
    SELECT 1
    FROM Orders o
    WHERE o.EmployeeID = e.EmployeeID
);
```

Joins

Theory:

- **INNER JOIN**: Matches rows in both tables.
- **LEFT JOIN**: Includes all rows from the left table.
- **RIGHT JOIN**: Includes all rows from the right table.
- **FULL JOIN**: Includes all rows from both tables.

Example:

```
SELECT c.FirstName, c.LastName, o.OrderDate
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Table Expressions

Theory:

- **Common Table Expression (CTE)**: Temporary result set for cleaner queries.
- **Derived Tables**: Subqueries in the FROM clause.

Example (CTE):

```
WITH SalesCTE AS (  
    SELECT ProductID, SUM(UnitPrice) AS TotalSales  
    FROM Sales  
    GROUP BY ProductID  
)  
SELECT p.ProductName, s.TotalSales  
FROM Products p  
JOIN SalesCTE s ON p.ProductID = s.ProductID;
```

Set Operators

Theory:

- **UNION**: Combines distinct rows.
- **INTERSECT**: Returns common rows.
- **EXCEPT**: Returns rows in the first query but not the second.

Example:

```
SELECT CustomerID FROM Customers  
UNION  
SELECT CustomerID FROM Orders;
```

Views

Theory: Views are virtual tables based on a query, simplifying data access.

Example:

```
CREATE VIEW HighValueCustomers AS  
SELECT CustomerID, FirstName, LastName  
FROM Customers  
WHERE TotalPurchases > 1000;  
  
SELECT * FROM HighValueCustomers;
```

Stored Procedures

Theory: Stored procedures are reusable T-SQL scripts for complex operations.

Example:

```
CREATE PROCEDURE GetEmployeeDetails
    @Department NVARCHAR(50)
AS
BEGIN
    SELECT FirstName, LastName, Salary
    FROM Employees
    WHERE Department = @Department;
END;

EXEC GetEmployeeDetails @Department = 'Sales';
```

Querying Metadata

Theory: System catalog views (e.g., INFORMATION_SCHEMA) provide database metadata.

Example:

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';
```

Classwork 9.1:

1. Write a query using INNER JOIN to combine Customers and Orders , grouping by CustomerID to count orders per customer.

```
SELECT c.CustomerID, c.FirstName, c.LastName, COUNT(o.OrderID) AS OrderCount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName;
```

2. Create a view for this query.

```
CREATE VIEW CustomerOrderCount AS
SELECT c.CustomerID, c.FirstName, c.LastName, COUNT(o.OrderID) AS OrderCount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName;
```

Test for Sessions 7–9

1. Azure Account Creation:

- List the steps to create an Azure account and deploy a free-tier Azure SQL Database.

2. Azure SQL Connection:

- Describe how to connect to an Azure SQL database using SSMS, including firewall configuration.

3. SELECT Query:

- Write a query to retrieve products from `AdventureWorks2022` where `ListPrice > 500`, ordered by `Name`.

```
SELECT Name, ListPrice
FROM Production.Product
WHERE ListPrice > 500
ORDER BY Name ASC;
```

4. Joins:

- Write a LEFT JOIN query to list all customers and their orders (including customers without orders) from `Customers` and `Orders`.

```
SELECT c.FirstName, c.LastName, o.OrderDate
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

5. Subquery:

- Write a query to find employees whose salary is above the average salary in their department.

```

SELECT FirstName, LastName, Salary
FROM Employees e
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees e2
    WHERE e2.Department = e.Department
);

```

6. Stored Procedure:

- Create a stored procedure to retrieve products by category, accepting a `CategoryID` parameter.

```

CREATE PROCEDURE GetProductsByCategory
    @CategoryID INT
AS
BEGIN
    SELECT ProductName, ListPrice
    FROM Products
    WHERE ProductCategoryID = @CategoryID;
END;

```

7. Metadata:

- Write a query to list all tables in the `AdventureWorks2022` database.

```

SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_TYPE = 'BASE TABLE';

```

Deliverables Mapping: SQL Server - The Definitive Guide (SG, XP, TG - Sessions 7, 8, 9 & 10)